Sandia National Laboratories

Exceptional service in the national interest

OPTIMIZING STRATEGIES FOR MULTI-GPU APPLICATIONS

SC-2026

Simon Garcia de Gonzalo

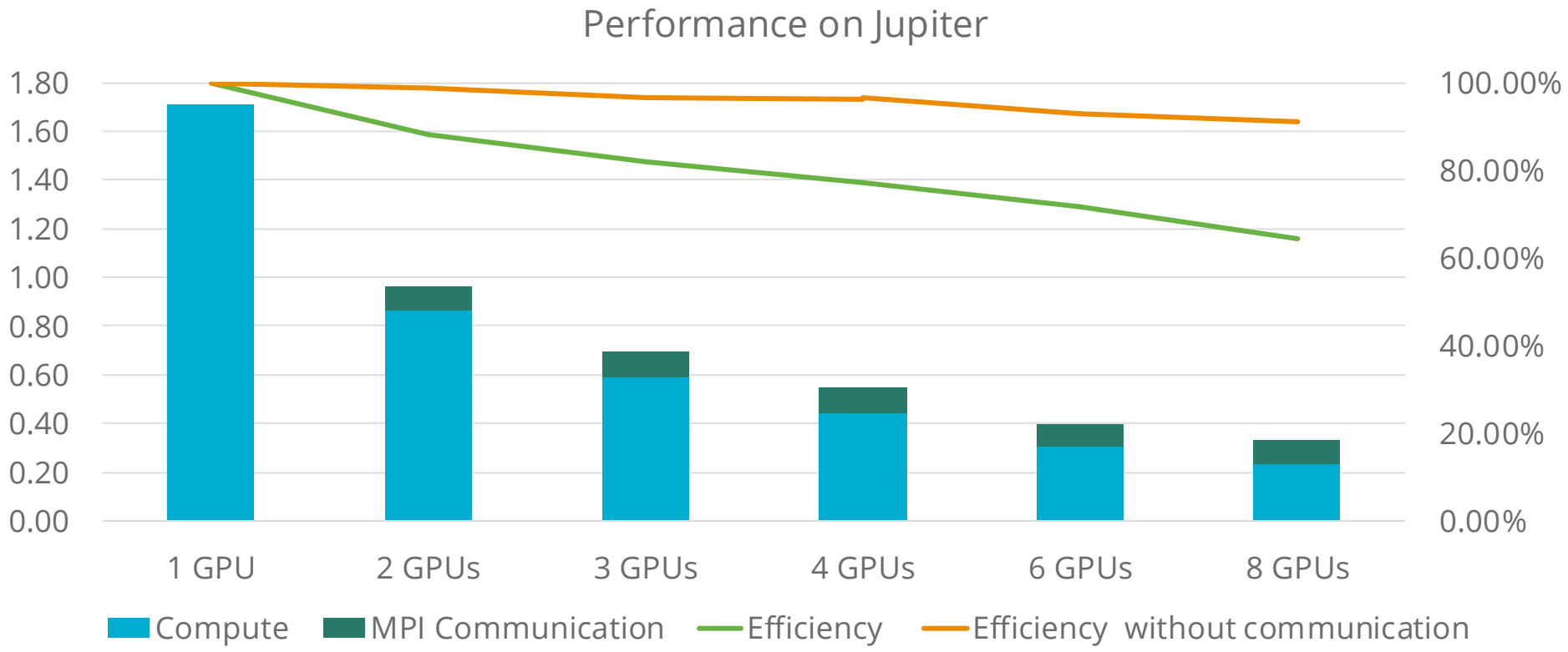
Slides provided by Lena Oden (FernUniversität in Hagen)

11/15/2026

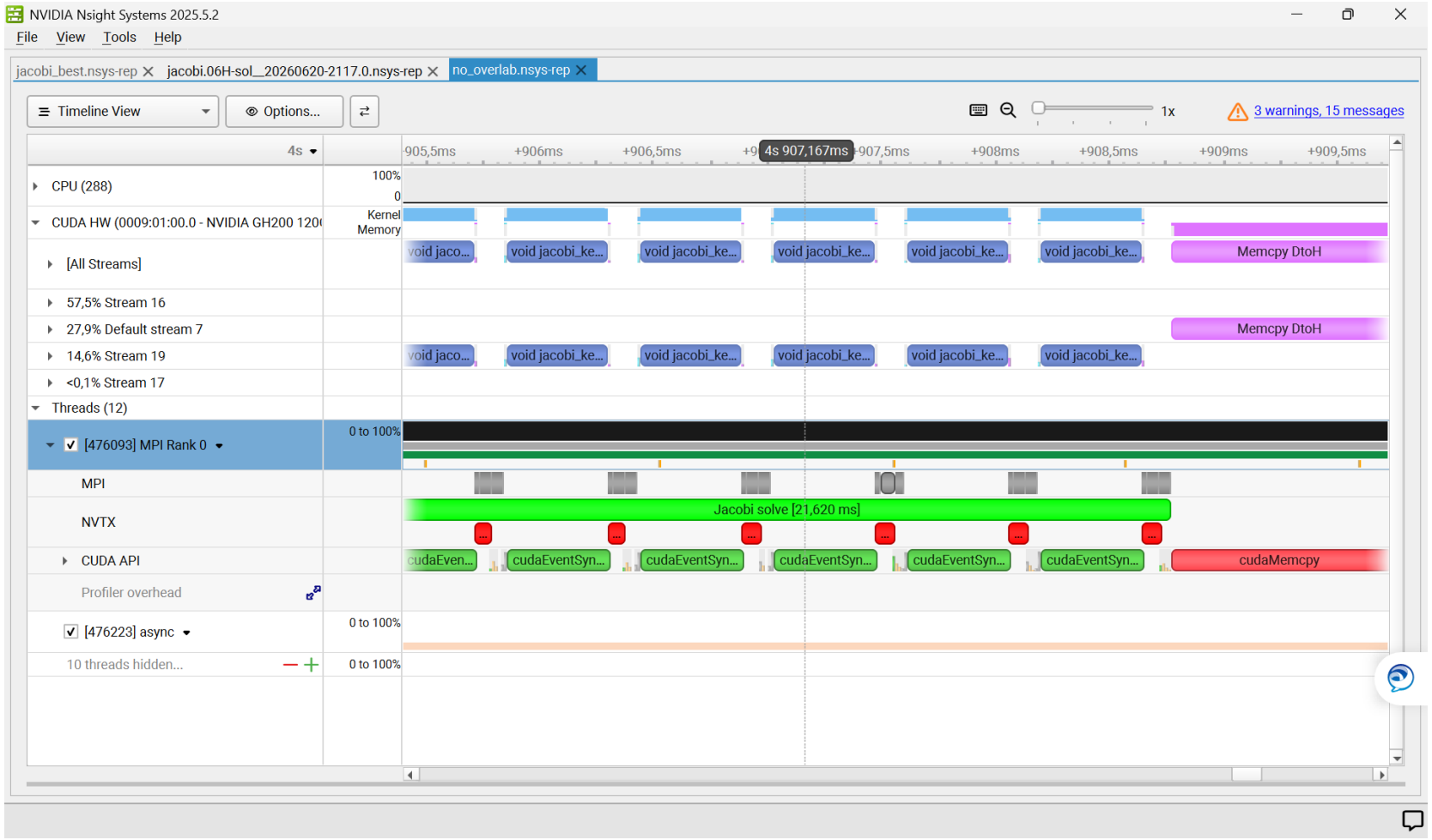
U.S. DEPARTMENT of ENERGY | NNSA National Nuclear Security Administration

Sandia National Laboratories is a multimission laboratory managed and operated by National Technology & Engineering Solutions of Sandia, LLC, a wholly owned subsidiary of Honeywell International Inc., for the U.S. Department of Energy's National Nuclear Security Administration under contract DE-NA0003525.

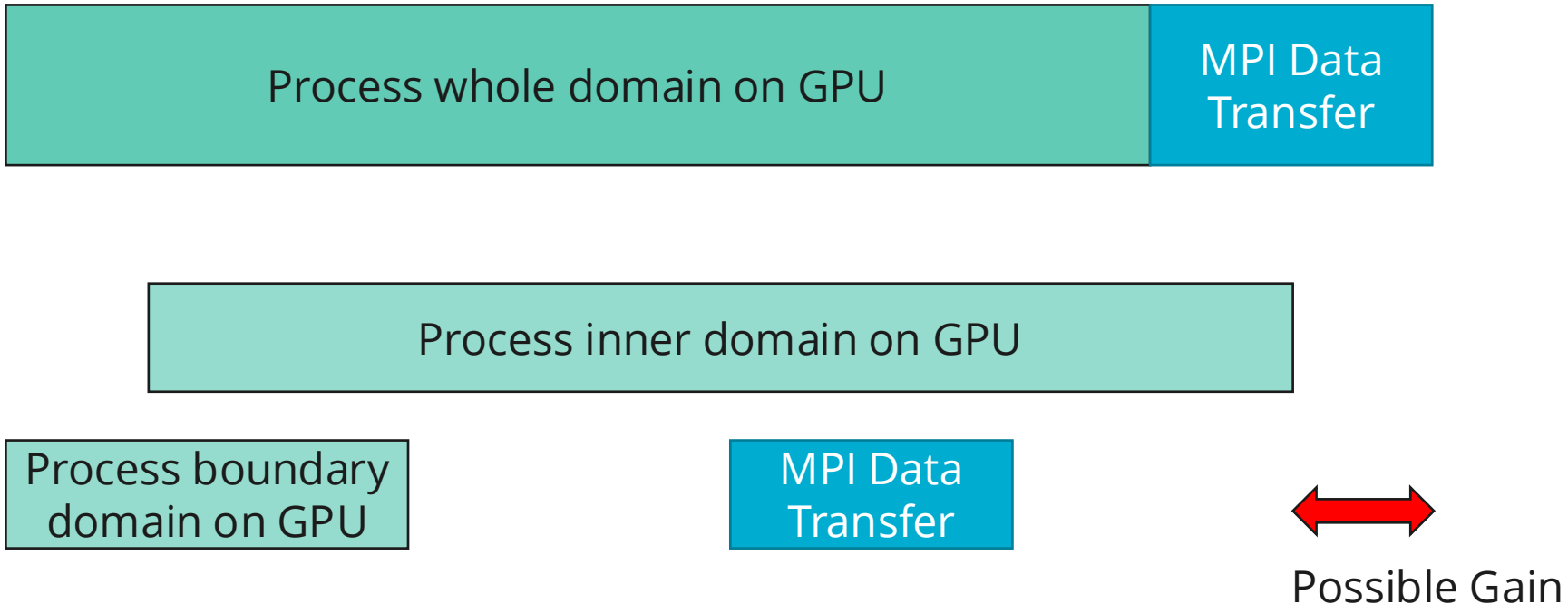
PERFORMANCE OF OUR FIRST MULTI-GPU APPLICATION



ANALSYSIS WITH NSIGHT SYSTEM



OVERLAPPING COMMUNICATION AND COMPUTATION



SHORT INTRO: CUDA-STREAMS

- sequence of operations (such as kernel executions, memory copies, and events) that execute in a specific order on the GPU
- operations within a single stream are guaranteed to execute sequentially, operations in different streams can execute concurrently (simultaneously) or overla
- By default, if you don't specify a stream, CUDA uses the "Default (Null) Stream." This stream is synchronous: it blocks operations in all other streams until it finishes, preventing concurrency.

```
cudaStream_t top_stream;

cudaStreamCreate(&top_stream);


…


cudaStreamSynchronize(&top_stream);
```

OVERLAPPING COMMUNICATION AND COMPUTATION

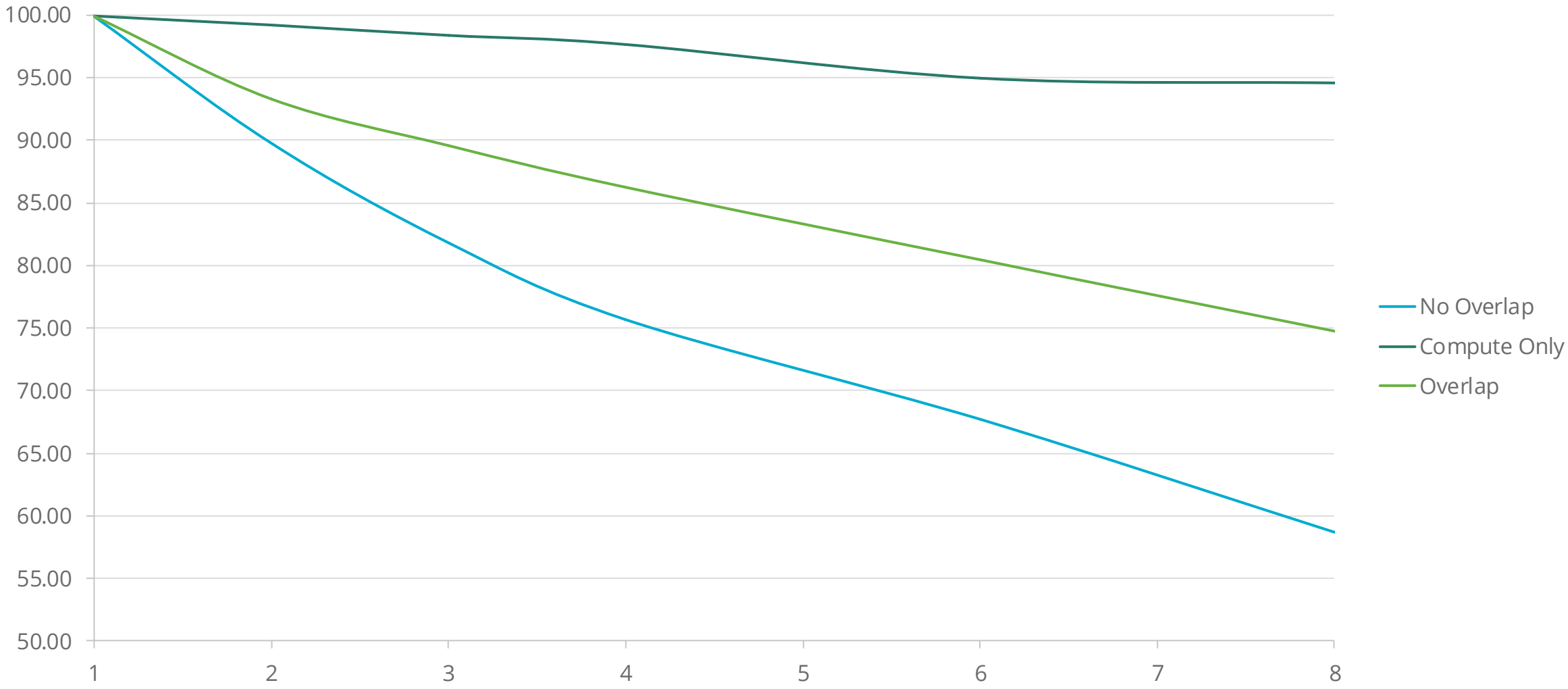
```
launch_jacobi_kernel(a_new, a, l2_norm_d, iy_start, (iy_start + 1), nx,
top_stream); launch_jacobi_kernel(a_new, a, l2_norm_d, (iy_end - 1), iy_end, nx,
bottom_stream); launch_jacobi_kernel(a_new, a, l2_norm_d, (iy_start + 1), (iy_end -
1), nx, compute_stream); const int top = rank > 0 ? rank - 1 : (size - 1);
const int bottom = (rank + 1) % size;

cudaStreamSynchronize(push_top_stream)

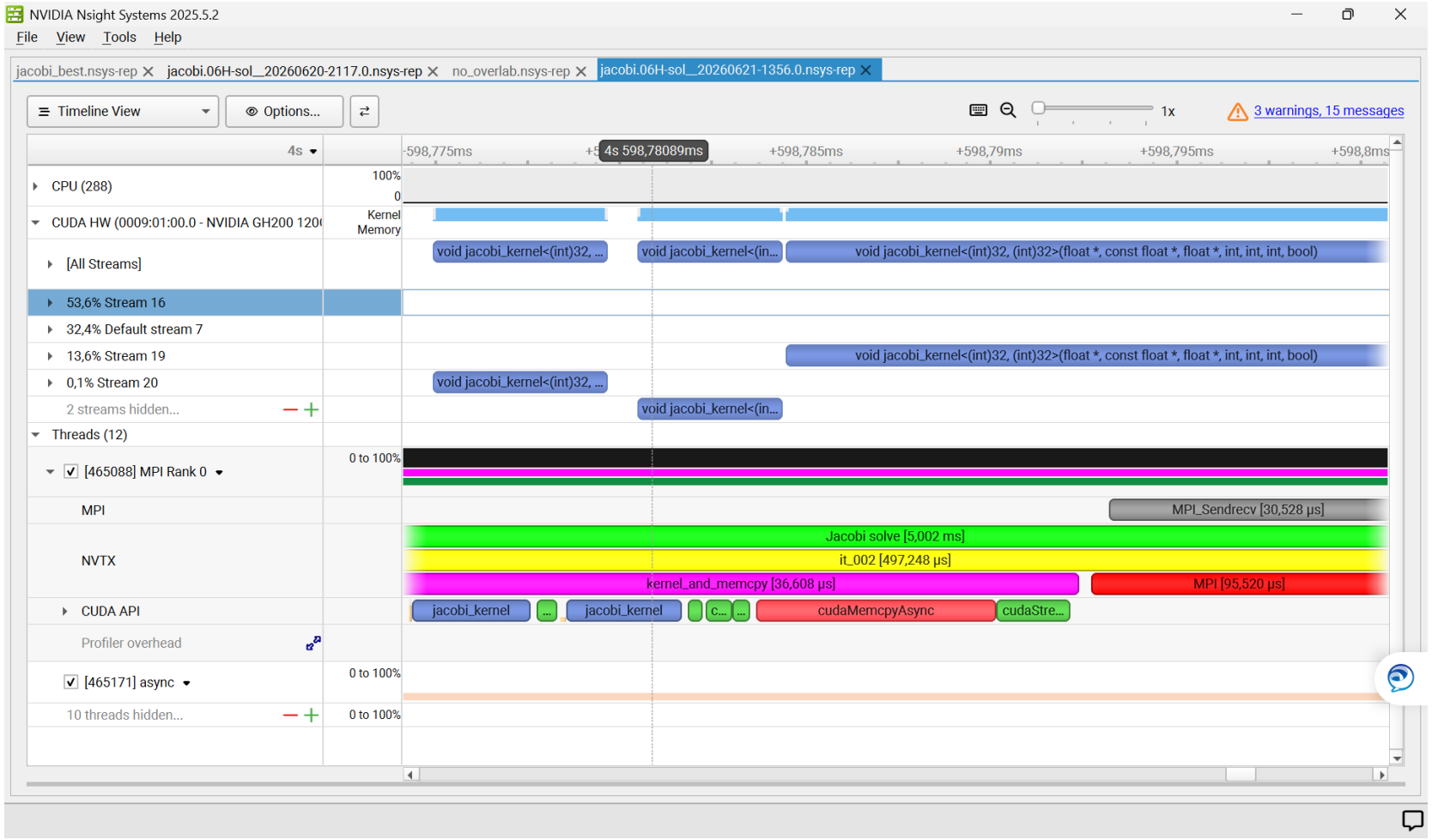
MPI_Sendrecv(a_new + iy_start * nx, nx, MPI_REAL_TYPE, top, 0, a_new + (iy_end *
nx), nx,      MPI_REAL_TYPE, bottom, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
cudaStreamSynchronize(push_bottom_stream));

MPI_Sendrecv(a_new + (iy_end - 1) * nx, nx, MPI_REAL_TYPE, bottom, 0, a_new, nx,
       MPI_REAL_TYPE, top, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

PERFORMANCE: COMMUNICATION + COMPUTATION OVERLAP, JUPITER, OPENMPI



OVERLAPPING COMMUNICATION AND COMPUTATION



OVERLAPPING MPI COMMUNICATION AND COMPUTATION

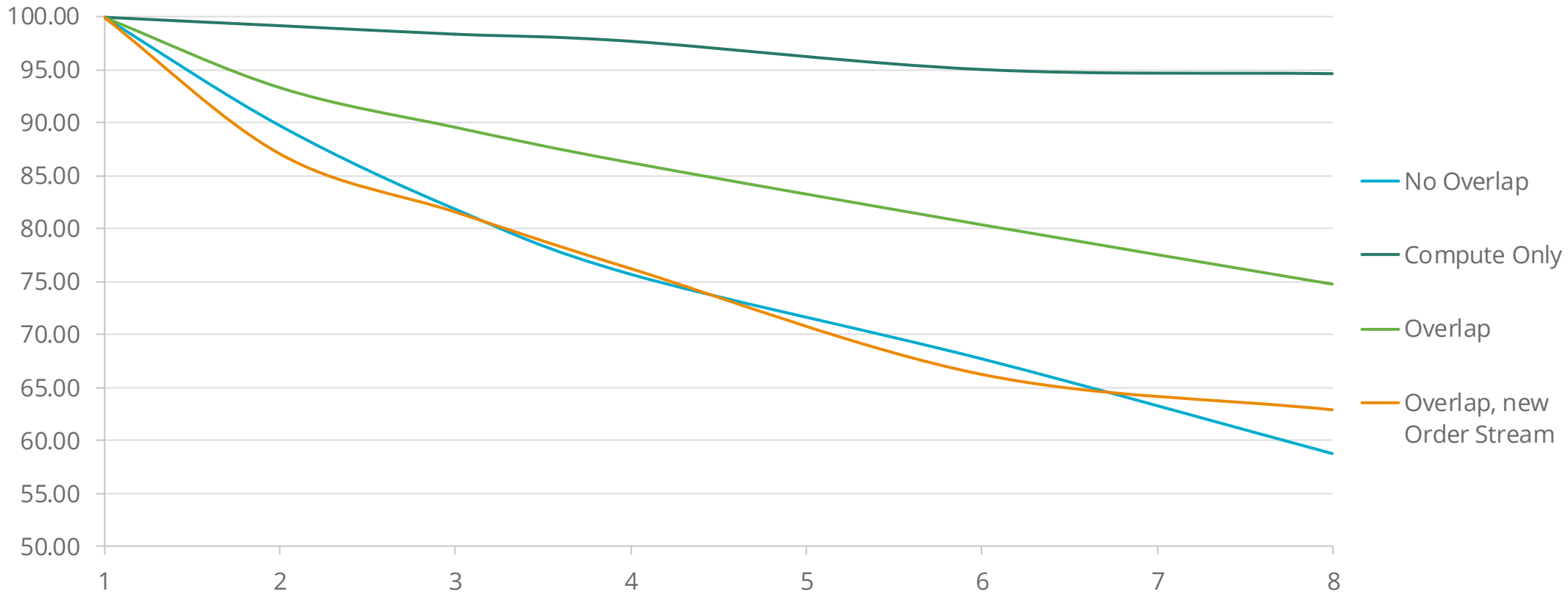
```
launch_jacobi_kernel(a_new, a, l2_norm_d, (iy_start + 1), (iy_end - 1), nx, compute_stream);
launch_jacobi_kernel(a_new, a, l2_norm_d, iy_start, (iy_start + 1), nx, push_top_stream);
launch_jacobi_kernel(a_new, a, l2_norm_d, (iy_end - 1), iy_end, nx, push_bottom_stream);
launch_jacobi_kernel(a_new, a, l2_norm_d, (iy_start + 1), (iy_end - 1), nx, compute_stream);
const int top = rank > 0 ? rank - 1 : (size - 1);
const int bottom = (rank + 1) % size;

cudaStreamSynchronize(push_top_stream)

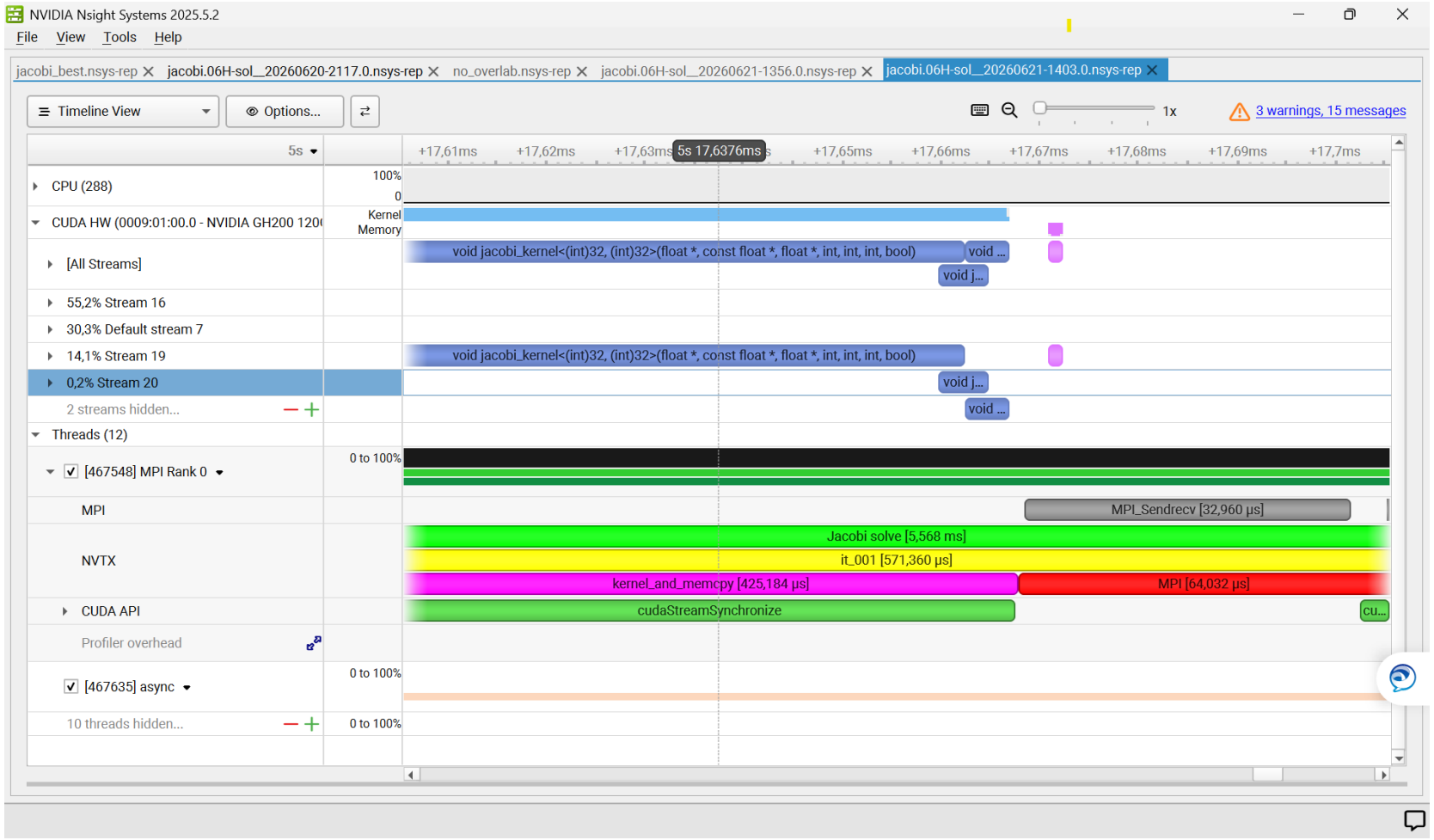
MPI_Sendrecv(a_new + iy_start * nx, nx, MPI_REAL_TYPE, top, 0, a_new + (iy_end * nx), nx,
        MPI_REAL_TYPE, bottom, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
cudaStreamSynchronize(push_bottom_stream));

MPI_Sendrecv(a_new + (iy_end - 1) * nx, nx, MPI_REAL_TYPE, bottom, 0, a_new, nx,
        MPI_REAL_TYPE, top, 0, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

PERFORMANCE: COMMUNICATION + COMPUTATION OVERLAP, JUPITER, OPENMPI



OVERLAPPING COMMUNICATION AND COMPUTATION



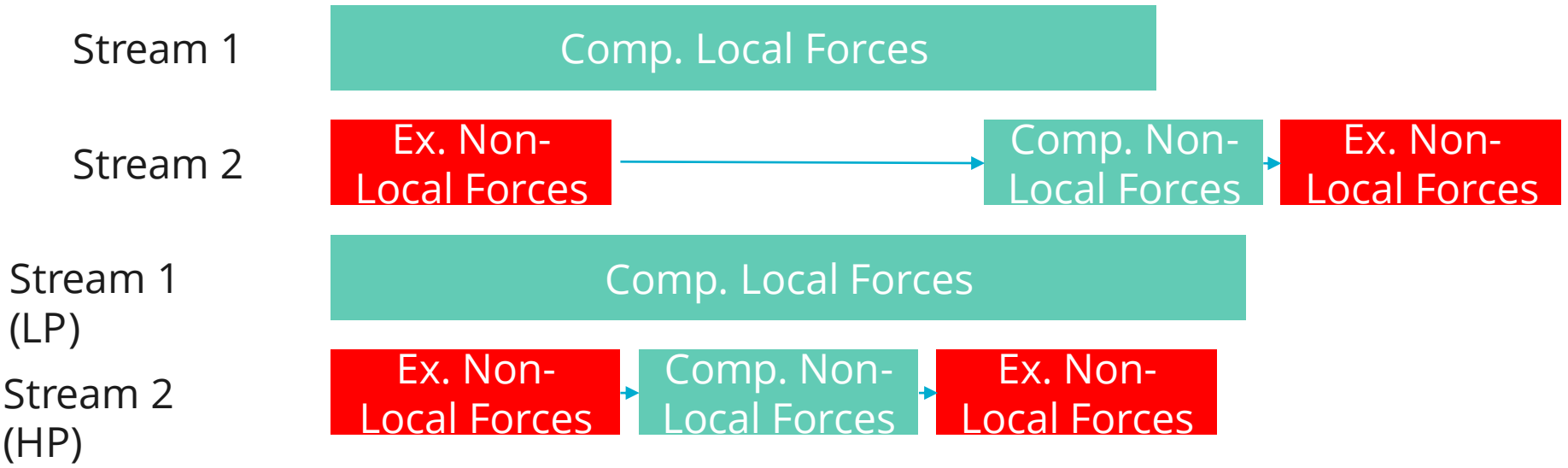
HIGH PRIORITY STREAMS

Improve scalability with high priority streams (available on CC 3.5+)

```
cudaStreamCreateWithPriority ( cudaStream_t* pStream, unsigned int flags, int priority )

cudaError_t cudaDeviceGetStreamPriorityRange(int *leastPriority, int *greatestPriority)
```

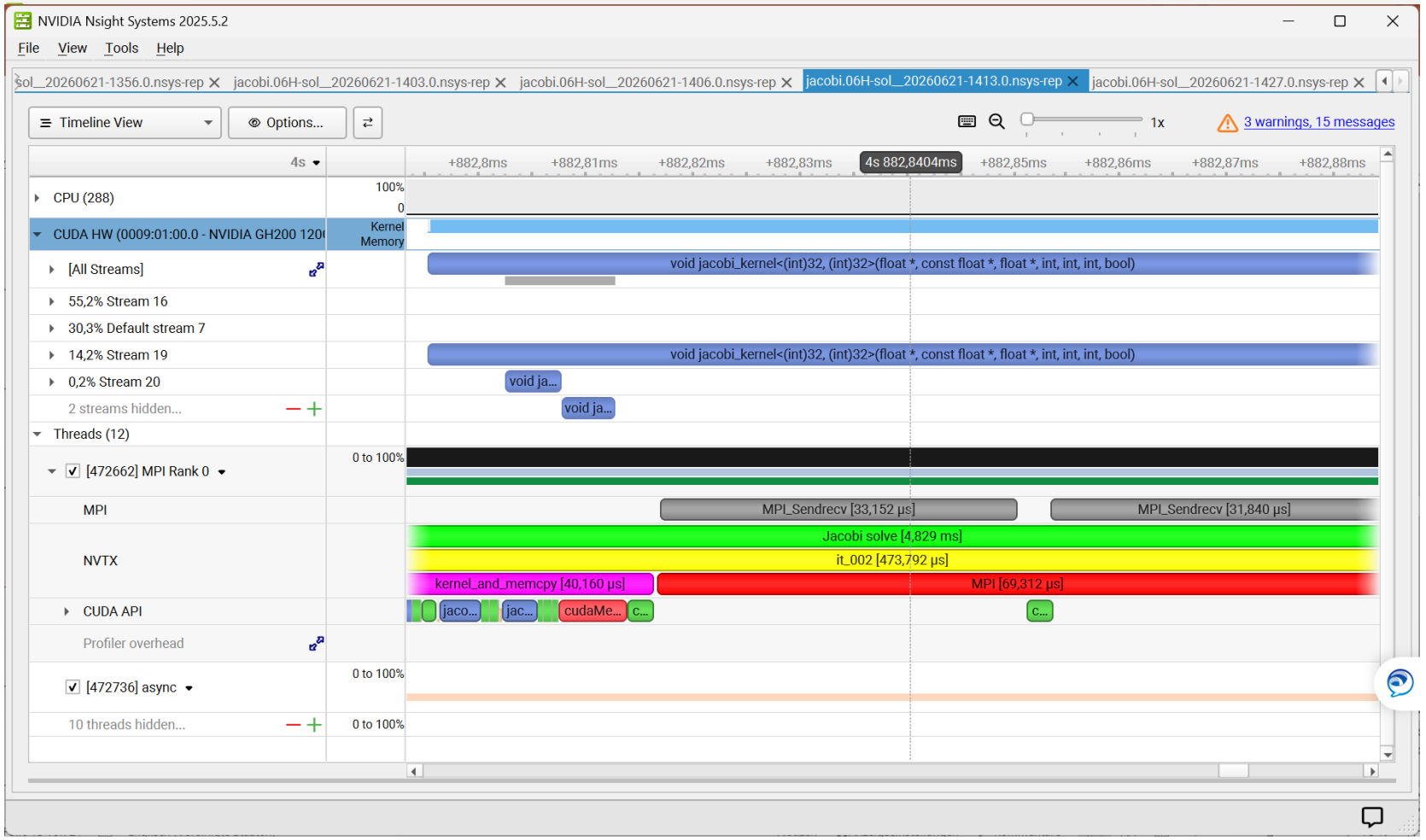
Use Cas: MD-Simulations



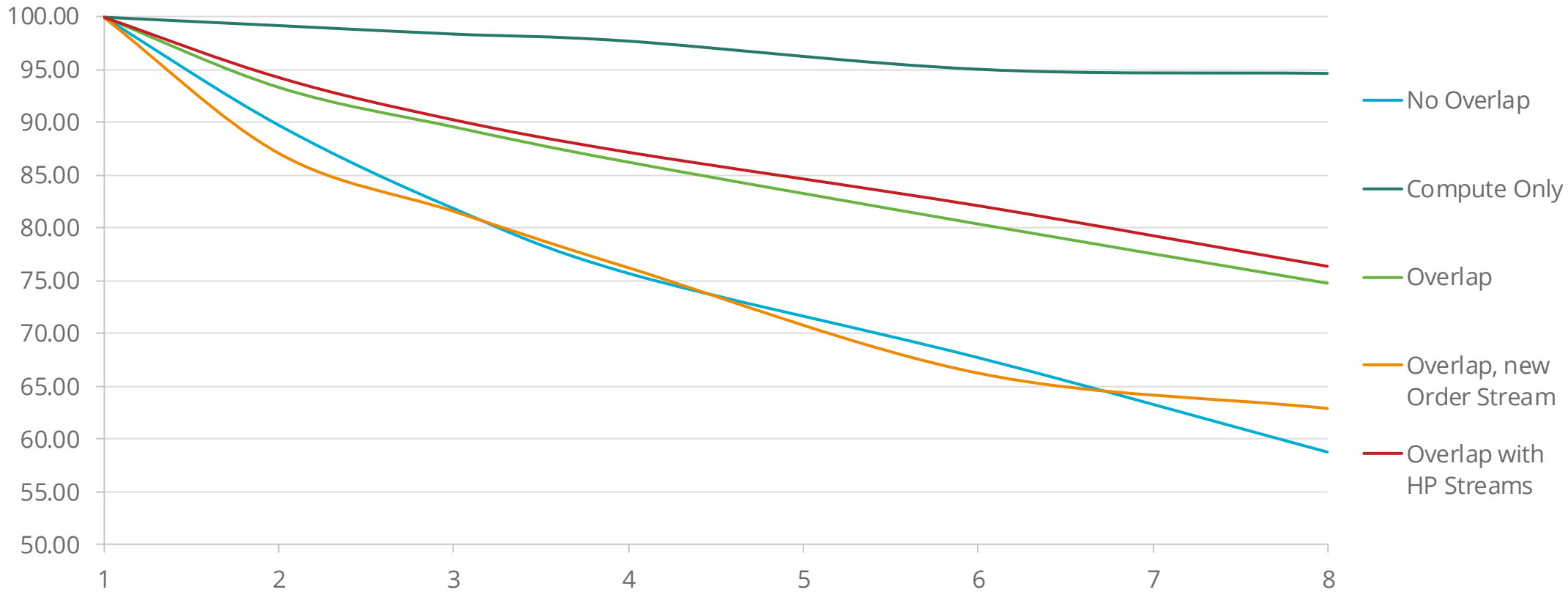
MPI COMMUNICATION + COMPUTATION OVERLAP WITH HIGH PRIORITY STREAMS

```
int leastPriority = 0;
int greatestPriority = leastPriority;
CUDA_RT_CALL(cudaDeviceGetStreamPriorityRange(&leastPriority, &greatestPriority));
cudaStream_t compute_stream;
cudaStream_t push_top_stream;
cudaStream_t push_bottom_stream;
CUDA_RT_CALL(cudaStreamCreateWithPriority(&compute_stream, cudaStreamDefault, leastPriority));
CUDA_RT_CALL(cudaStreamCreateWithPriority(&push_top_stream, cudaStreamDefault, greatestPriority));
CUDA_RT_CALL(cudaStreamCreateWithPriority(&push_bottom_stream, cudaStreamDefault, greatestPriority));
```

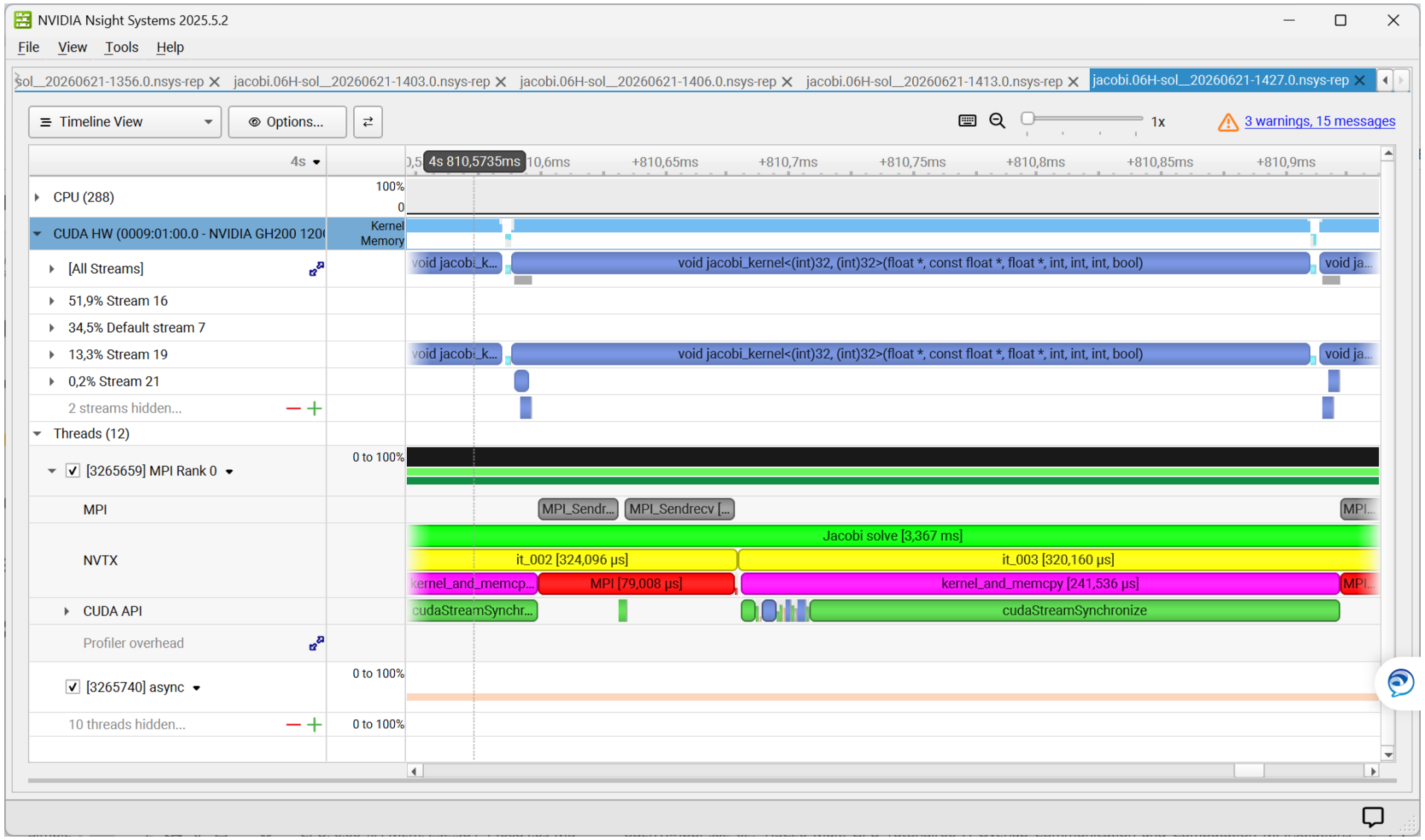
OVERLAP WITH HIGH PRIORITY STREAMS



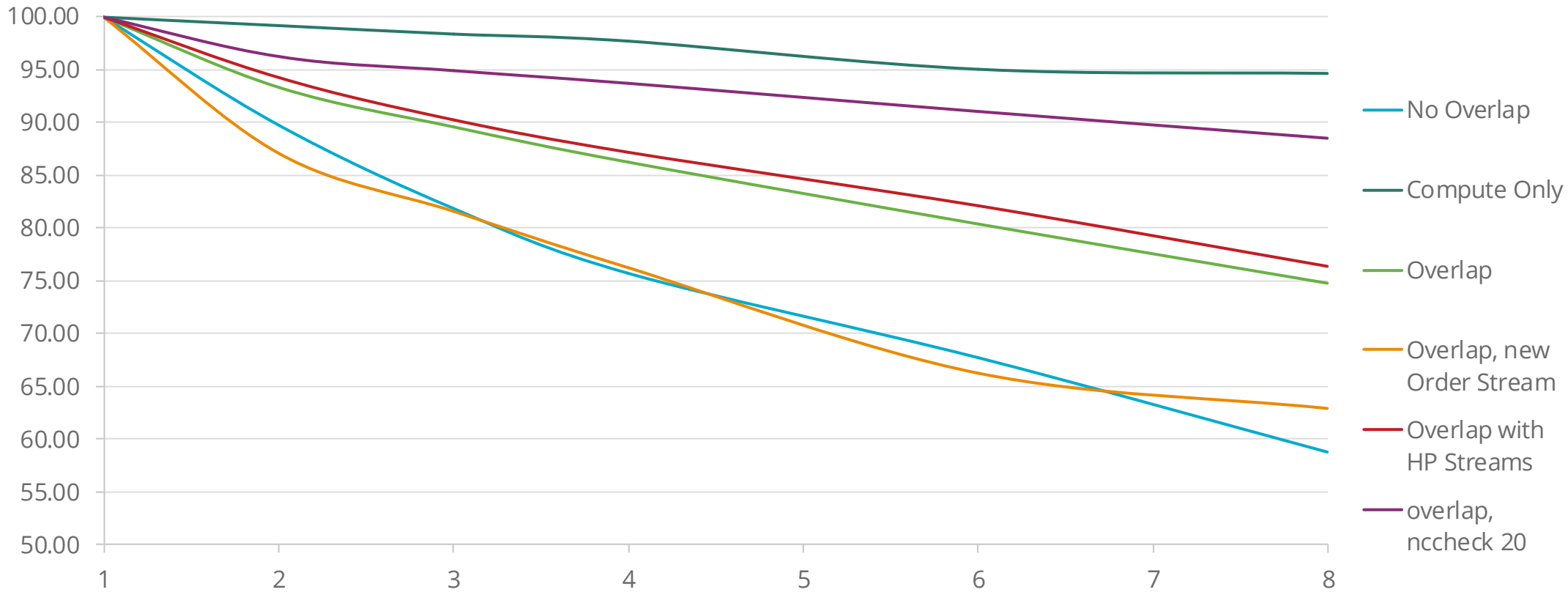
PERFORMANCE: COMMUNICATION + COMPUTATION OVERLAP, JUPITER, OPENMPI



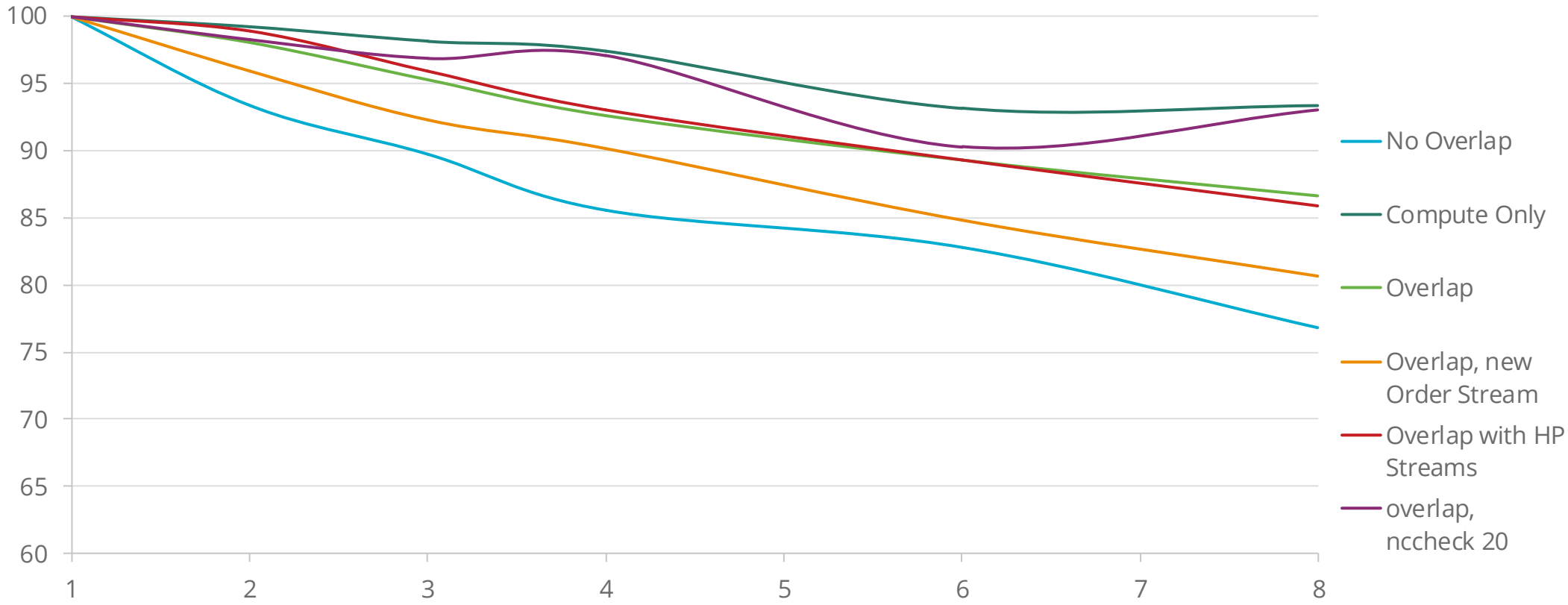
REMOVE THE ALLREDUCE-CALL WITH –NCCHECK N



PERFORMANCE: COMMUNICATION + COMPUTATION OVERLAP, JUPITER, OPENMPI



PERFORMANCE: COMMUNICATION + COMPUTATION OVERLAP, JUWELS BOOSTER, OPENMPI



CUDA-AWARE MPI

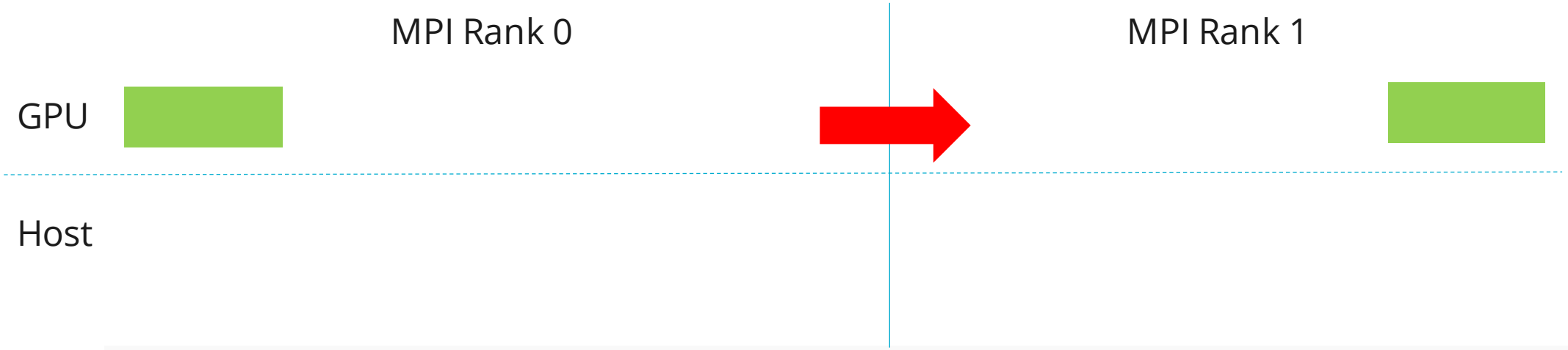
Example:

MPI Rank 0 MPI_Send from GPU Buffer

MPI Rank 1 MPI_Recv to GPU Buffer

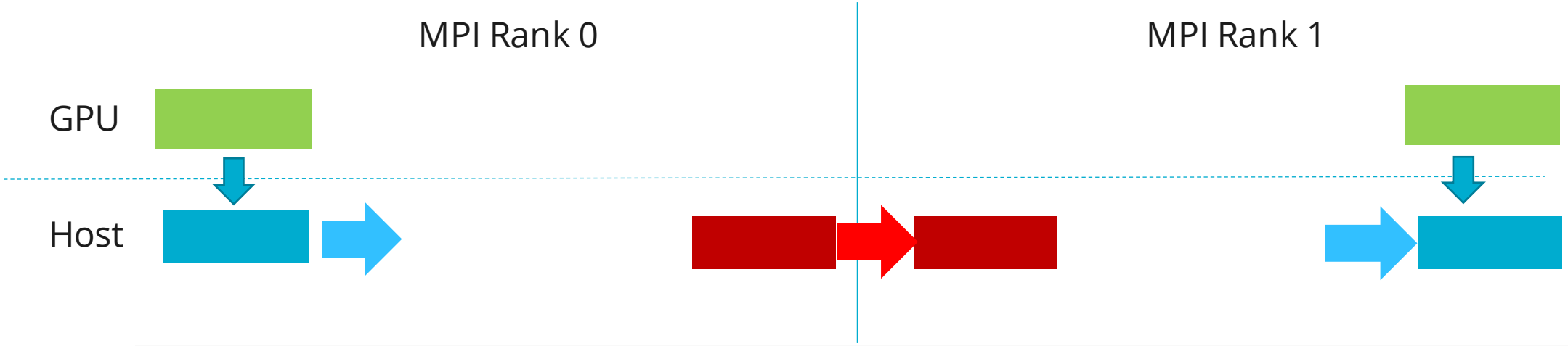
- Show how CUDA+MPI works in principle
- Depending on the MPI implementation, message size, system setup, Situation might be different
- Two GPUs in two nodes

GPU TO REMOTE GPU
CUDA-AWARE MPI WITH SUPPORT FOR GPUDIRECT RMA



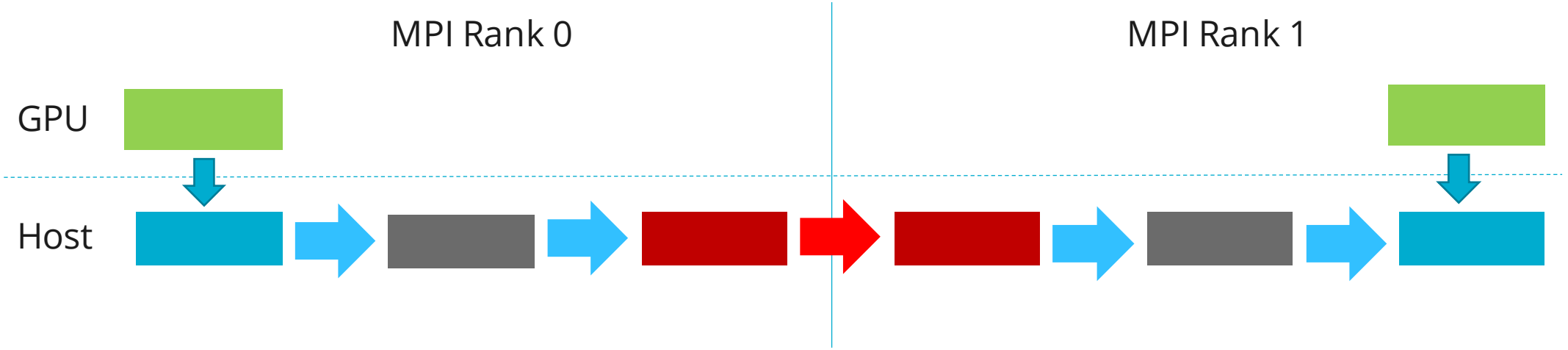

```
MPI_Send(s_buf_d, size, MPI_BYTE, 1, tag, MPI_COMM_WORLD);
MPI_Recv(r_buf_d, size, MPI_BYTE, 0, tag, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

GPU TO REMOTE GPU
CUDA-AWARE MPI WITHOUT SUPPORT FOR GPUDIRECT RMA



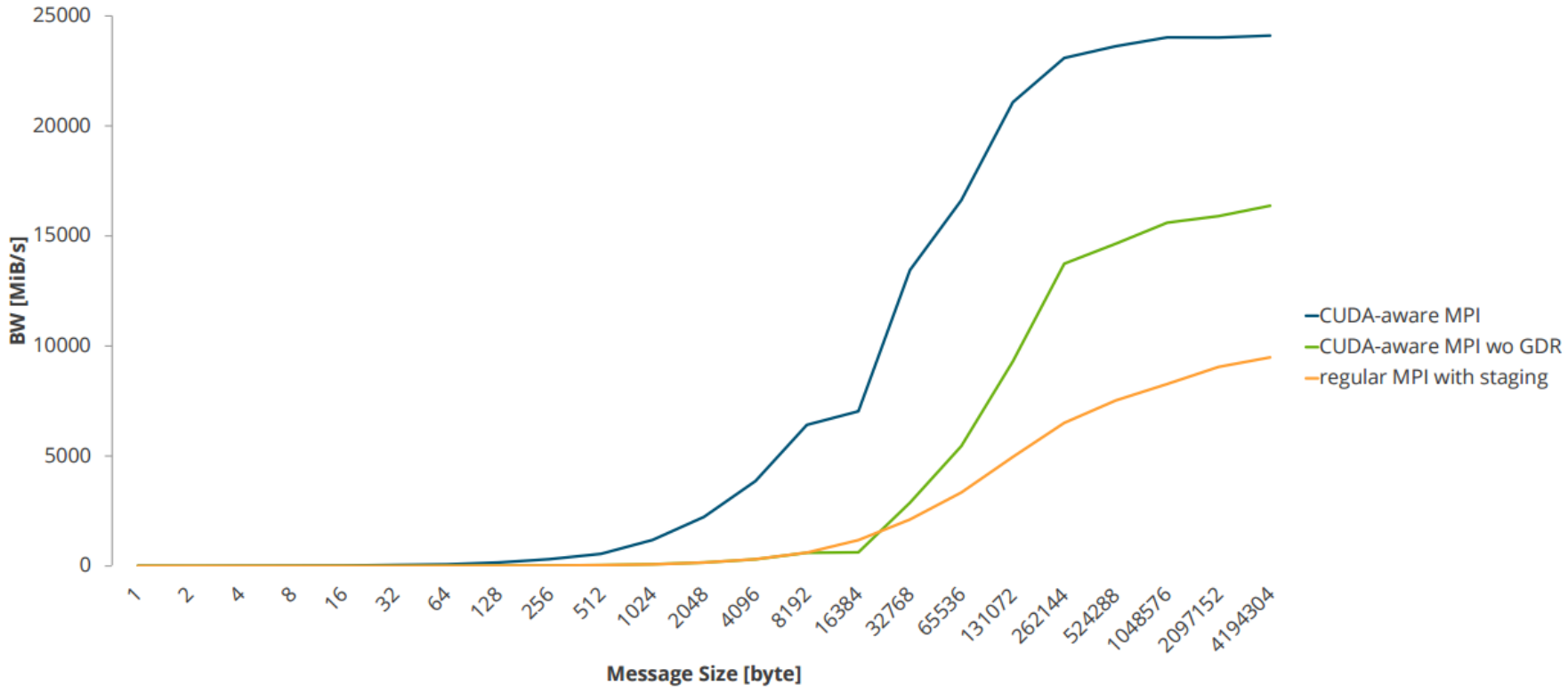

```
MPI_Send(s_buf_d, size, MPI_BYTE, 1, tag, MPI_COMM_WORLD);
MPI_Recv(r_buf_d, size, MPI_BYTE, 0, tag, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
```

GPU TO REMOTE GPU WITHOUT CUDA-AWARE MPI




```
cudaMemcpy(s_buf_h, s_buf_d, size, cudaMemcpyDeviceToHost);
MPI_Send(s_buf_d, size, MPI_BYTE, 1, tag, MPI_COMM_WORLD);
MPI_Recv(r_buf_d, size, MPI_BYTE, 0, tag, MPI_COMM_WORLD, MPI_STATUS_IGNORE);
cudaMemcpy(r_buf_d, r_buf_h, size, cudaMemcpyHostToDevice);
```

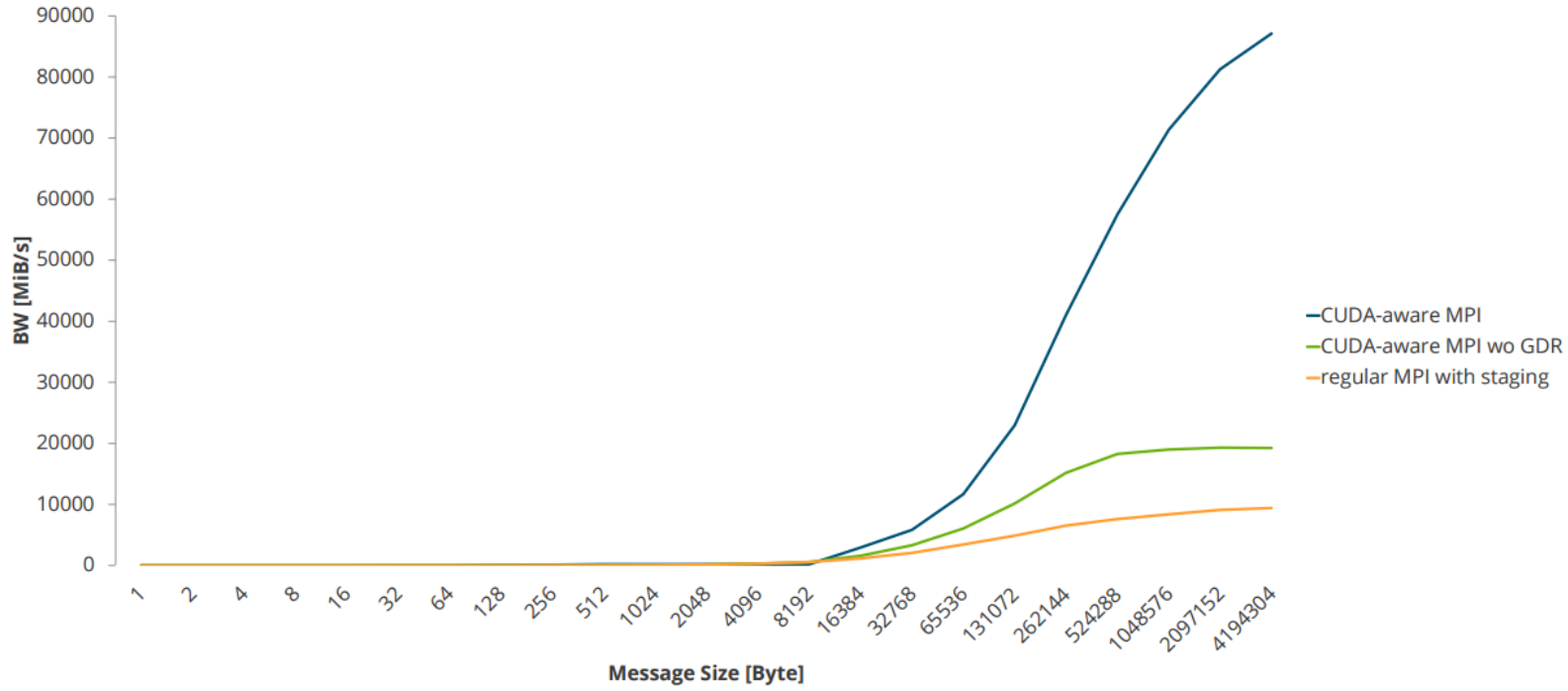
PERFORMANCE RESULTS GPUDIRECT RDMA
OPEN MPI 4.1.0RC1 + UCX 1.9.0 ON JUWELS BOOSTER



Latency (1 byte) 4.27 us 24.56 us 25.64 us

JUWELS Booster: https://apps.fz-juelich.de/jsc/hps/juwels/booster-overview.html

PERFORMANCE RESULTS GPUDIRECT P2P
OPEN MPI 4.1.0RC1 + UCX 1.9.0 ON JUWELS BOOSTER

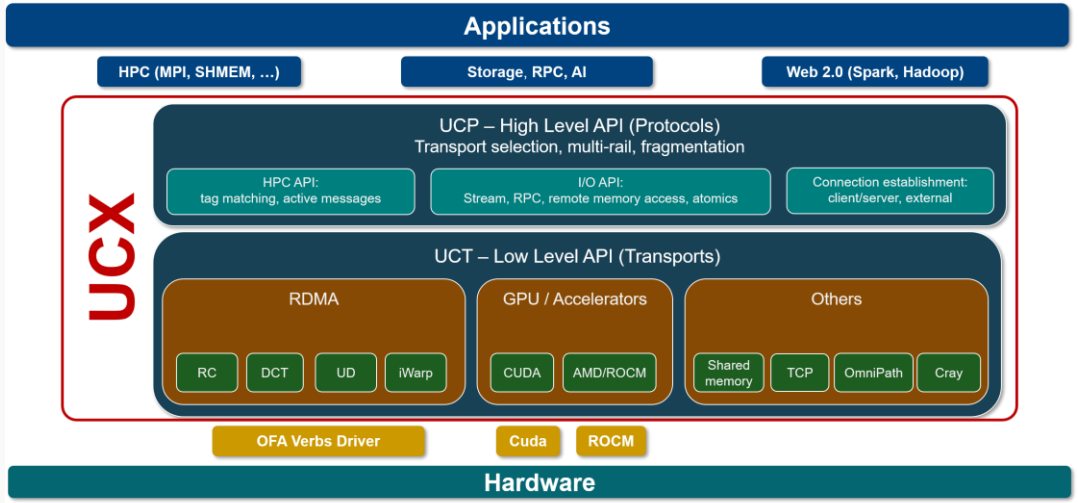


Latency (1 byte) 2.45 us 22.01 us 23.50 us

JUWELS Booster: https://apps.fz-juelich.de/jsc/hps/juwels/booster-overview.html

UCX TIPS AND TRICKS

```
$ ucx_info -caf | grep -B9 UCX_RNDV_SCHEME
#
# Communication scheme in RNDV protocol.
# get_zcopy - use get_zcopy scheme in RNDV protocol.
# put_zcopy - use put_zcopy scheme in RNDV protocol.
# auto - runtime automatically chooses optimal
scheme to use.
#
# syntax: [get_zcopy|put_zcopy|auto]
#
UCX_RNDV_SCHEME=auto
```



UCX TIPS AND TRICKS
ENABLE LOGGING TO SEE WHAT IS GOOING ON

`UCX_LOG_LEVEL=data UCX_LOG_FILE=log-%h-%p` *`//helpful to check for used protocols and selected HCAs`*`:`

[1605706306.970537] [jwb1238:7263 :0] ucp_worker.c:1627 UCX INFO ep_cfg[0]: tag(cuda_copy/cuda); rma(gdr_copy/cuda);

[1605706306.972721] [jwb1238:7263 :0] ucp_worker.c:1627 UCX INFO ep_cfg[1]: tag(self/memory rc_mlx5/mlx5_1:1 cma/memory cuda_copy/cuda);

[1605706306.997849] [jwb1238:7263 :1] ucp_worker.c:1627 UCX INFO ep_cfg[2]: tag(rc_mlx5/mlx5_1:1);

UCX TIPS AND TRICKS

UCX_NET_DEVICES: To select HCA for optimal GPU-HCA affinity, should not be necessary with UCX 1.9 or newer

UCX_TLS: Select transports to use, default: all

cuda is an alias for: cuda_copy, cuda_ipc, gdr_copy

To run without any GPUDirect flavor set UCX_TLS to only include cuda_copy from the CUDA related transports, e.g. UCX_TLS=rc,sm,cuda_copy and UCX_IB_GPU_DIRECT_RDMA=no (rc transport uses GPUDirect RDMA otherwise).

Parastation MPI also has PSP_CUDA_ENFORCE_STAGING=1.

UCX_MEMTYPE_CACHE: Set to n to disable mem type cache. Sometimes necessary if the CUDA runtime is linked statically!

SUMMARY

- Asynchronously computing on the GPU while MPI communication allows to hide MPI
- communication times
- Using high priority streams some CUDA API overheads can be also hidden
- GPUDirect can provide significant performance improvements both inter and intra node
- Knowing UCX performance tuning knobs is important when working with CUDA-aware
- MPI implementations like OpenMPI and Parastation MPI built on UCX.